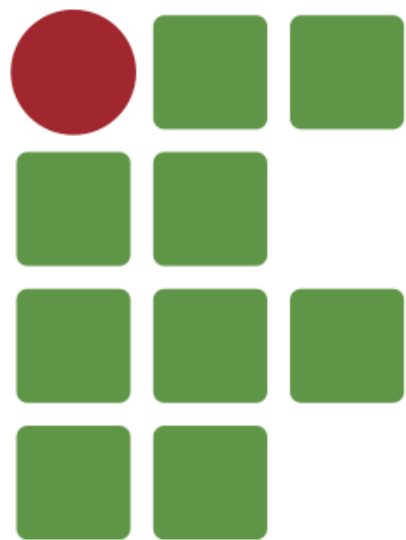




**INSTITUTO
FEDERAL**
Brasília

ARQUITETURA DE COMPUTADORES

Prof. MSc. Thiago Neves
IFB Campus Taguatinga
20/09/2026

AULA 06

Assembler MIPS – Parte 1



Conversão de Bases

Conversão de Bases (Binário para Decimal)

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
128	64	32	16	8	4	2	1	
1	1	0	0	0	1	1	0	
$128*1$	$64*1$	$32*0$	$16*0$	$8*0$	$4*1$	$2*1$	$1*0$	
128	+64	+0	+0	+0	+4	+2	+0	= 198

- Converta os seguintes números binários para decimal:

☐ 1010

☐ 110011

☐ 100000

☐ 11111111

Conversão de Bases (Decimal para Binário)

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	128	64	32	16	8	4	2	1
51 =	0	+0	+32	+16	+0	+0	+2	+1
	0	0	1	1	0	0	1	1

- Converta os seguintes números decimais para binário:

☐ 15

☐ 32

☐ 45

☐ 128

Conversão de Bases (Decimal para Hex)

26 em Hexadecimal?

$$26 \div 16 = 1 \text{ resto } 10$$

$$1 \div 16 = 0 \text{ resto } 1$$

= 1A

100 em Hexadecimal?

$$100 \div 16 = 6 \text{ resto } 4$$

$$6 \div 16 = 0 \text{ resto } 6$$

= 64

Decimal	Hex
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	A
11	B
12	C
13	D
14	E
15	F

Conversão de Bases (Decimal para Hex)

2026 em Hexadecimal?

$$2026 \div 16 = 126 \text{ resto } 10$$

$$126 \div 16 = 7 \text{ resto } 14$$

$$7 \div 16 = 0 \text{ resto } 7$$

= 7EA

Decimal	Hex
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	A
11	B
12	C
13	D
14	E
15	F

Representação Hexadecimal

Em C, um número hexadecimal é representado por:

`0x1234` = 4.660

2 números Hex::

$$16 \times 16 = 2^4 \times 2^4 = 2^8 = 1 \text{ byte}$$

4 números Hex:

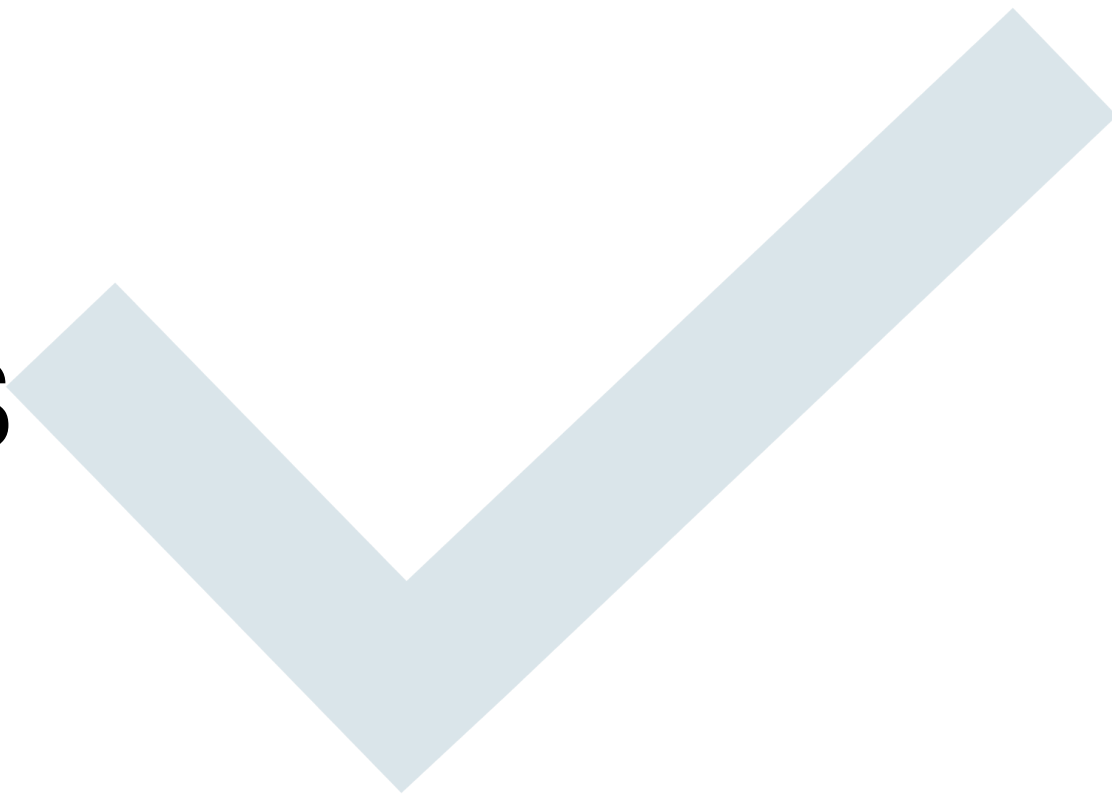
$$16 \times 16 \times 16 \times 16 = 2^8 \times 2^8 = 2^{16} = 2 \text{ bytes}$$

Endereço IPv6: 2001:0db8:85a3:0000:0000:8a2e:0370:7334

Quantos bytes? Quantos bits?



Assembler MIPS



Instruction Set Architecture (ISA)

Uma Arquitetura do Conjunto de Instruções (ISA) é uma especificação que descreve os aspectos do processador que são visíveis a um programador em linguagem de máquina (ou assembly), tais como:

- ❑ instruções;
- ❑ registradores;
- ❑ modelo de memória;
- ❑ entrada e saída (I/O);
- ❑ exceções;
- ❑ entre outros.

Instruction Set Architecture (ISA)

A mesma ISA pode ser implementada de diferentes maneiras.

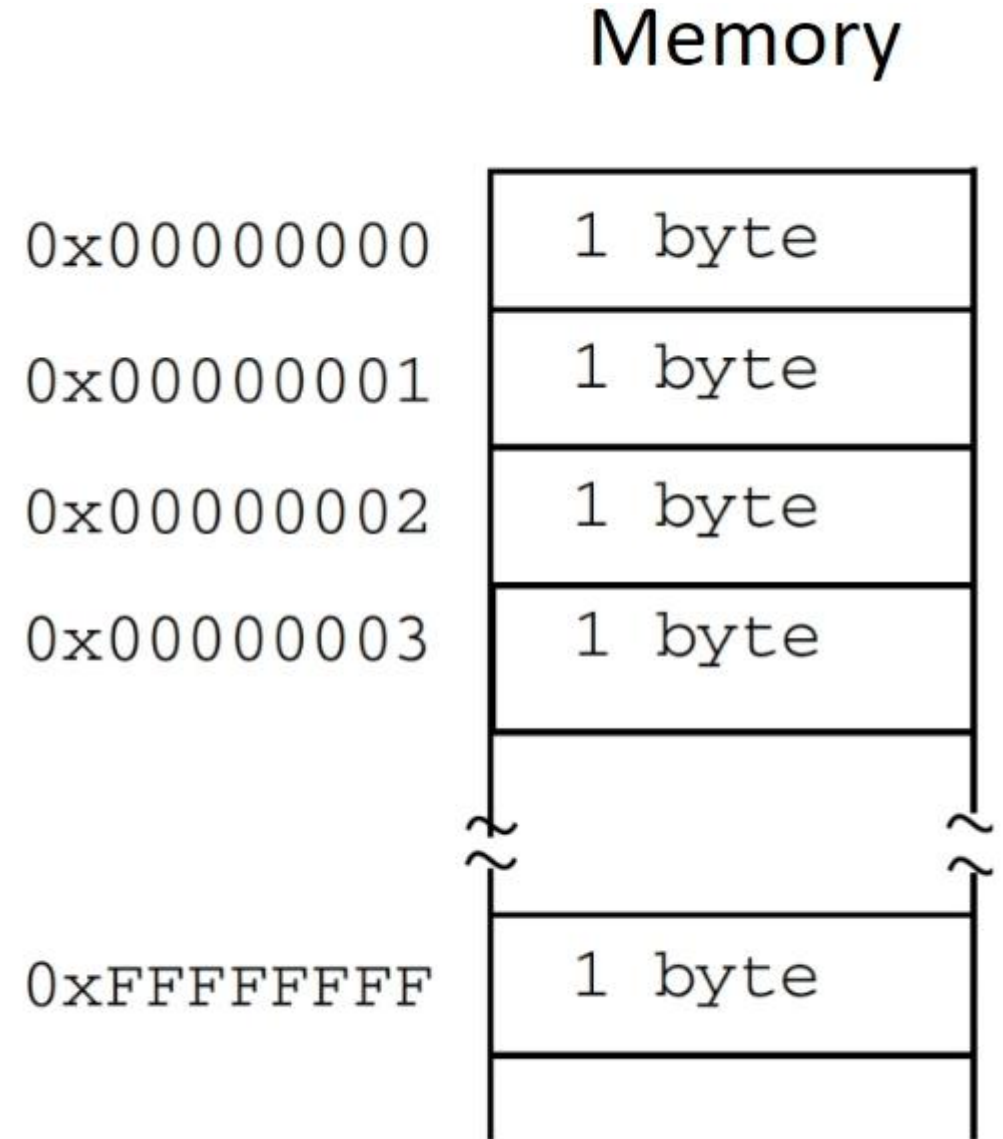
❑ Um programa compilado para uma determinada ISA pode ser executado em qualquer hardware que implemente essa ISA.

Exemplos: **MIPS**, RISC-V, x86, entre outras.

Memória no MIPS

Memória = vetor em que cada elemento é identificado por um endereço e contém 8 bits (1 byte).

2^{32} bytes endereçáveis.



Representação de Bytes

Little-endian: colocamos primeiro o byte **menos significativo** no endereço de número mais baixo.

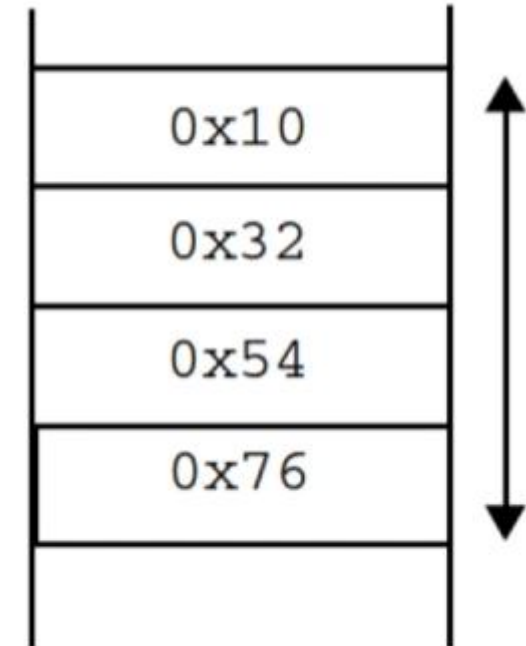
Exemplo: salvamos o número 0x76543210 no endereço 0x00010000.

0x00010000

0x00010001

0x00010002

0x00010003



Representação de Bytes

Big-endian: colocamos primeiro o byte **mais significativo** no endereço de número mais baixo.

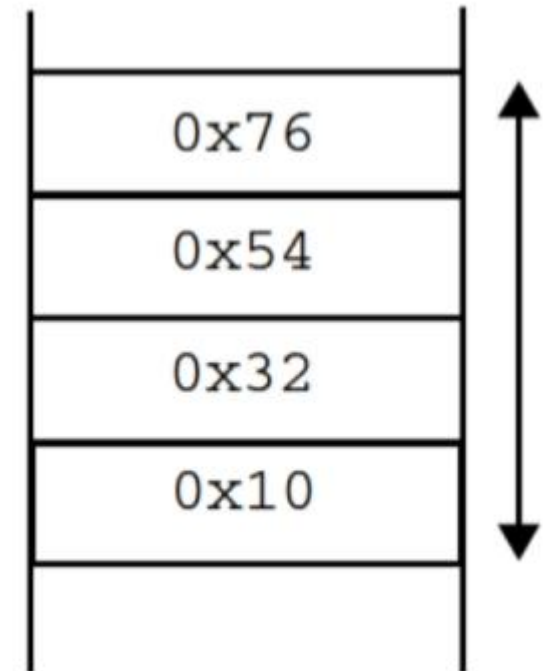
Exemplo: salvamos o número 0x76543210 no endereço 0x00010000.

0x00010000

0x00010001

0x00010002

0x00010003



Variáveis em C

```
1  int g1, g2; —————> Variáveis externas
2  void main() {
3      int l1, l2; —————> Variáveis internas
4      l1 = g1;
5      ...
6  }
```

MIPS: Variáveis locais e globais

- Var globais = Memória
- Var locais = Registradores

```
1  int g1, g2;  
2  void main() {  
3      int l1, l2;  
4      l1 = g1;  
5      ...  
6  }
```

Example MIPS:

```
1  .data  
2  g1: .word 0  
3  g2: .word 0  
4  .text  
5  .globl main  
6  main:  
7  la $t0, g1  
8  lw $t1, 0($t0)  # l1 = g1  
9  ...
```

MIPS: Variáveis

Tamanho das variáveis

- char -> 1 byte
- short -> 2 bytes
- int -> 4 bytes
- long long -> 8 bytes

Declaração:

- char -> .byte
- short -> .half
- int -> .word
- long long -> .dword

MIPS: Variáveis

Linguagem C

```
char c = 0x11;  
short s = 0x2211;  
int i = 0x44332211;  
unsigned int ui;  
long long l = 0x8877665544332211
```

Linguagem MIPS

```
        .data  
c: .byte 0x11  
s: .half 0x2211  
i: .word 0x44332211  
ui: .word 0  
l: .dword 0x8877665544332211
```

MIPS: Alinhamento das variáveis

O endereço inicial de uma variável deve ser múltiplo do tamanho da variável.

Por padrão, variáveis globais são localizadas em endereços múltiplos de seu tamanho.

.byte -> x1

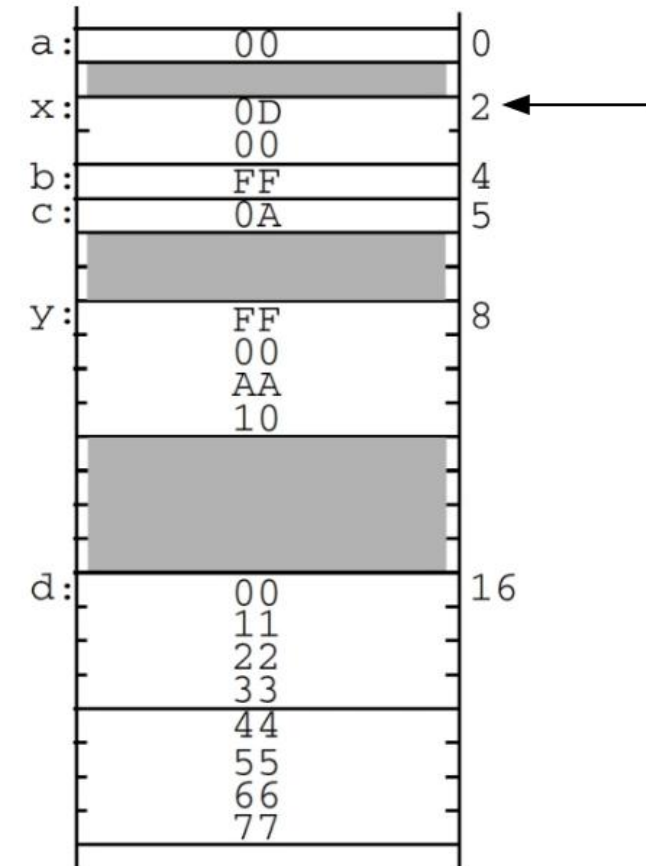
.half -> x2

.word -> x4

.dword -> x8

MIPS: Alinhamento das variáveis

```
1 a: .byte 0
2 x: .half 13
3 b: .byte -1
4 c: .byte 10
5 y: .word 0x10AA00FF
6 d: .dword 0x7766554433221100
```



MIPS: Alinhamento de vetores

```
int v = {1, 2, 3, 4, 5};
```

```
v: .word 1, 2, 3, 4, 5
```

```
char v[100];
```

```
v: .space 100
```

(Reserva 100 bytes com valor 0)

MIPS: Alinhamento de vetores

Align (assembler directive)

.align n: coloca o próximo dado em um endereço que é múltiplo de 2^n .

```
1  char c;  
2  int v[100];
```

```
1  c:  .byte 0  
2      .align 2  
3  v:  .space 100*4
```

MIPS: Registradores de propósito geral

No **MIPS32**, há **32 registradores de propósito geral (GPRs)**, cada um com **32 bits**.

\$zero → 0

\$v0-\$v1 → valores de retorno

\$a0-\$a3 → argumentos

\$t0-\$t9 → temporários

\$s0-\$s7 → valores que devem ser preservados

\$sp → pilha (stack)

\$fp → frame da função

\$ra → endereço de retorno

MIPS: Todos os registradores

Number	Name	Utilization
\$0	\$zero	It is always zero, not modifiable
\$1	\$at	Reserved for macro expansion
\$2-\$3	\$v0-\$v1	Result of subroutines
\$4-\$7	\$a0-\$a3	Subroutine arguments
\$8-\$15	\$t0-\$t7	Unpreserved temporals for subroutine calls
\$16-\$23	\$s0-\$s7	Preserved temporals for subroutine calls
\$24-\$25	\$t8-\$t9	Unpreserved temporals for subroutine calls
\$26-\$27	\$k0-\$k1	Reserved for the OS (should not be used)
\$28	\$gp	Global pointer
\$29	\$sp	Stack pointer, contains the address of the top of the stack.
\$30	\$fp	Frame pointer
\$31	\$ra	Return address

MIPS: Operandos

- ❑ Maior parte das instruções: 3 operandos.
- ❑ **Modo de Endereçamento (addressing mode)**: maneira como o operando é especificado.
- ❑ Tipos de modo de endereçamento:
 - ❑ Registro
 - ❑ Imediato
 - ❑ Memória
 - ❑ Pseudo-direto
 - ❑ Relativo ao PC

MIPS: Operandos

(Com registradores)

`addu $t1, $t2, $t3` # $\$t1 = \$t2 + \$t3$

(Com imediatos)

`addiu $t1, $t2, 10` # $\$t1 = \$t2 + 10$

MIPS: Soma e Subtração

addu rd, rs, rt

$rd = rs + rt$

addiu rt, rs, imm16

$rt = rs + \text{SignExt}(\text{imm16})$

subu rd, rs, rt

$rd = rs - rt$

**SignExt(imm16) =
Complemento a 2 com 16 bits**

Complemento a 2 com 16 bits

É uma forma de representar **números inteiros positivos e negativos em binário.**

Com **16 bits**, podemos representar:

$$-2^{15} \leq x \leq 2^{15}-1$$

$$-32768 \leq x \leq +32767$$

Como representar 5?

$$5 = 101$$

$$= 0000\ 0000\ 0000\ 0101$$

(16 bits)

Como representar 32767?

$$32767 =$$

$$0111\ 1111\ 1111\ 1111$$

Complemento a 2 com 16 bits

Sendo:

$$5 = 0000\ 0000\ 0000\ 0101$$

Como representar -5?

(Fazer o complemento a 2 e somar +1)

$$-5 = 1111\ 1111\ 1111\ 1010 + 0000\ 0000\ 0000\ 0001$$

$$-5 = 1111\ 1111\ 1111\ 1011$$

Complemento a 2 com 16 bits

E somar 5 – 5?

0000 0000 0000 0101 + 1111 1111 1111 1011

= 1 0000 0000 0000 0000 (resultado = 17 bits, bit descartado)

= 0

Exemplo

```
1 void main () {  
2     int f, g, h, i, j;  
3     ...  
4     f = (g + h) - (i + j);  
5     ...
```

In MIPS, assuming f, g, h, i, j occupy \$t0, \$t1, \$t2, \$t3, \$t4:

```
1 addu $t0, $t1, $t2 # $t0 = g + h  
2 addu $t1, $t3, $t4 # $t1 = i + j  
3 subu $t0, $t0, $t1 # f = (g + h) - (i + j)
```

MIPS: Cópia de registradores

`addiu $t1, $t2, 0`

(Alternativa)

`addu $t1, $zero, $t2`

(Alternativa – pseudo instrução)

`move $t1, $t2`

MIPS: li macro

Se o operando for de 32bits (por exemplo, um endereço), podemos usar a seguinte macro:

```
li $t1, 0x44332211
```

(MARS)

```
li $t1, 0xFFFF = li $t1, 0x0000FFFF
```

MIPS: la macro

`.data`

`a: .word 0`

`.text`

`.globl main`

`main:`

`la $t0, a`

`.data` = Indica ao assembler que, a partir dali, estamos declarando **dados que ficarão na memória.**

`.text` = A partir daqui começa o código do programa.

`.globl main` = Isso informa ao assembler/linker que main é um **símbolo global.**

MIPS: Constantes Simbólicas

`.eqv N 10`

Em C: `#define N 10`

`.data`

`a: .word 0`

`.text`

`.globl main`

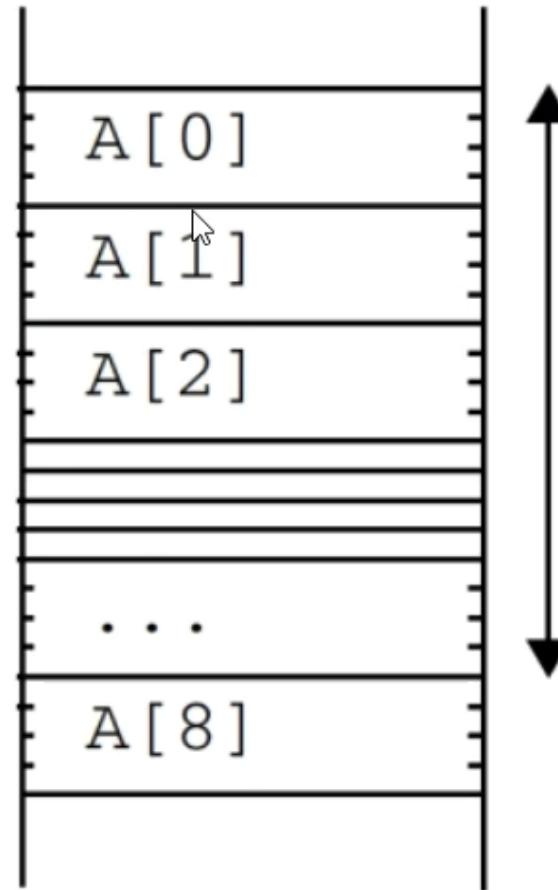
`main:`

`la $t0, a`

MIPS: Operandos em Memória

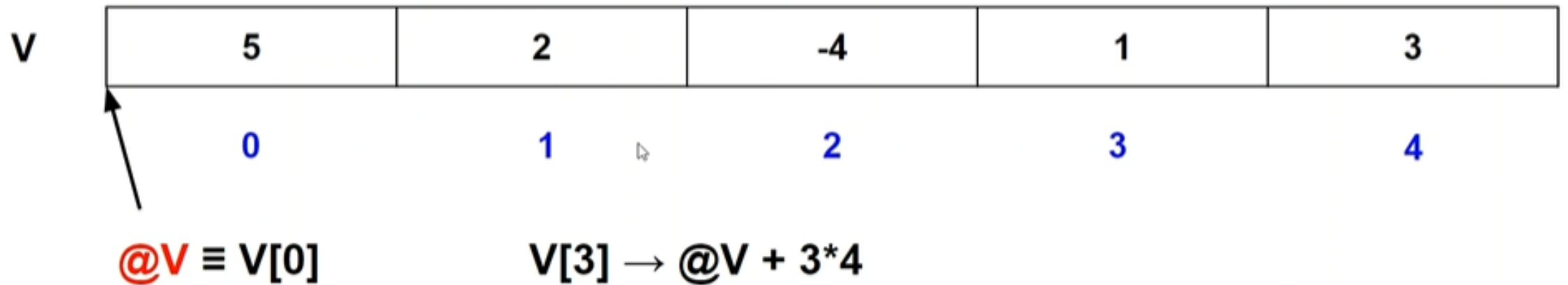
```
1  int A[100];  
2  void main() {  
3      int g, h; // $t1 , $t2.  
4      g = h + A[8];  
5  }
```

A:

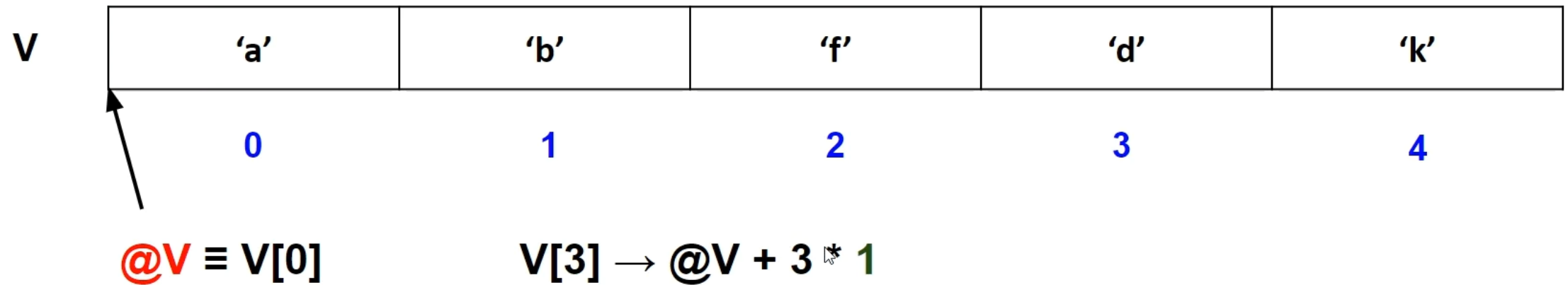


$\text{offset} = 8 * 4 = 32$

MIPS: Operandos em Memória



MIPS: Operandos em Memória



MIPS: Lendo da Memória

```
1  int A[100];  
2  void main() {  
3      int g, h; // $t1 , $t2.  
4      g = h + A[8];  
5  }
```

la \$t3, A

end do vetor

lw \$t0, 32(\$t3)

\$t0 = A[8]

addu \$t1, \$t2, \$t0

g = h + A[8]

MIPS: Escrevendo na Memória

```
1  int A[100];  
2  void main () {  
3      int h; // $t2.  
4      A[12] = h + A[8];  
5  }
```

```
la $t3, A           # end do vetor
```

```
lw $t0, 32($t3)     # $t0 = A[8]
```

```
addu $t0, $t2, $t0  # $t0 = h + A[8]
```

```
sw $t0, 48($t4)     # A[12] = h + A[8]
```

MIPS: Leitura e Escrita

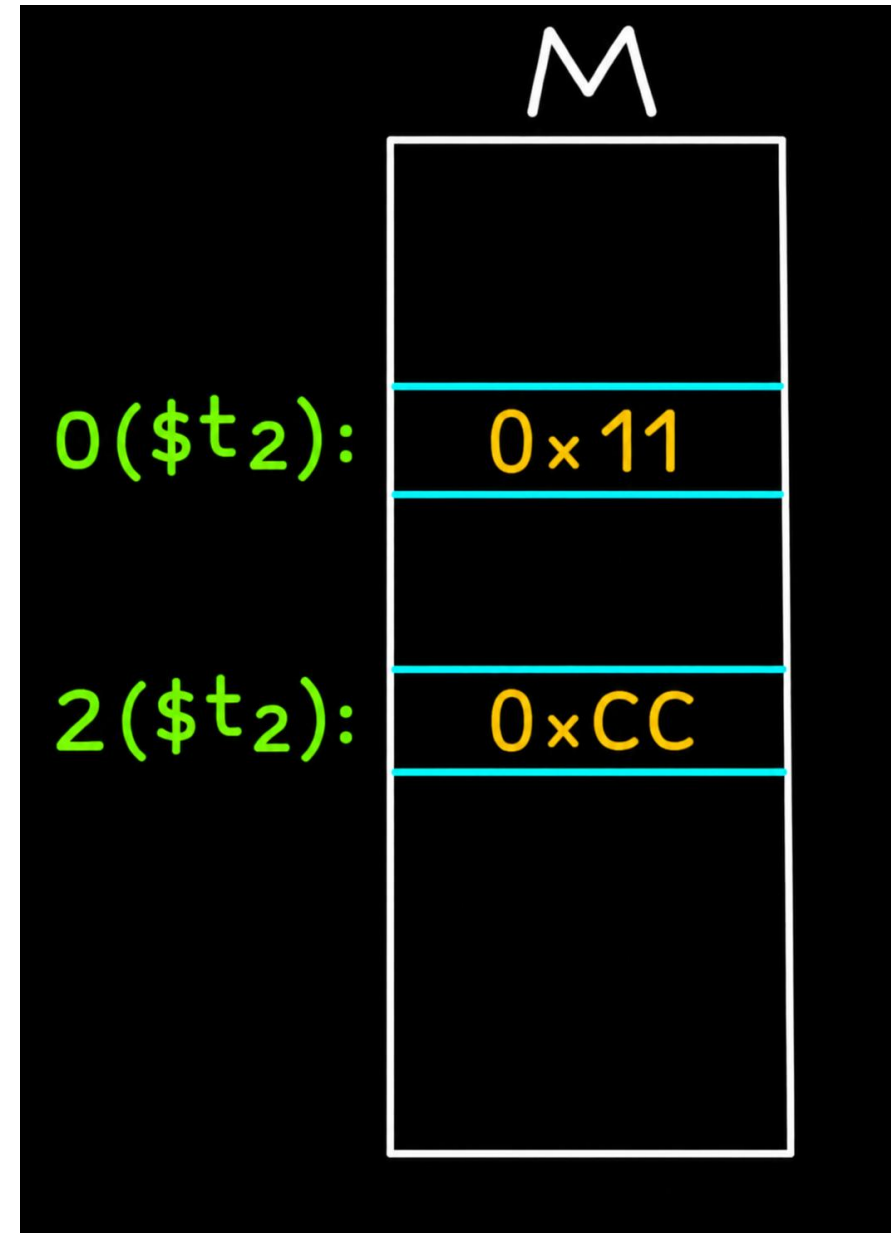
lw/lh/lhu/lb/lbu/sw/sh/sb			
lw	rt, off16(rs)	$rt = M_w[rs + \text{SignExt}(\text{off16})]$	load word
lh	rt, off16(rs)	$rt = \text{SignExt}(M_h[rs + \text{SignExt}(\text{off16})])$	load half (extends sign)
lhu	rt, off16(rs)	$rt = M_h[rs + \text{SignExt}(\text{off16})]$	load half (extends zeros)
lb	rt, off16(rs)	$rt = \text{SignExt}(M_b[rs + \text{SignExt}(\text{off16})])$	load byte (extends sign)
lbu	rt, off16(rs)	$rt = M_b[rs + \text{SignExt}(\text{off16})]$	load byte (extends zeros)
sw	rt, off16(rs)	$M_w[rs + \text{SignExt}(\text{off16})] = rt$	store word
sh	rt, off16(rs)	$M_h[rs + \text{SignExt}(\text{off16})] = rt_{15:0}$	store half
sb	rt, off16(rs)	$M_b[rs + \text{SignExt}(\text{off16})] = rt_{7:0}$	store byte

MIPS: lb x lbu (= lh x lhu)

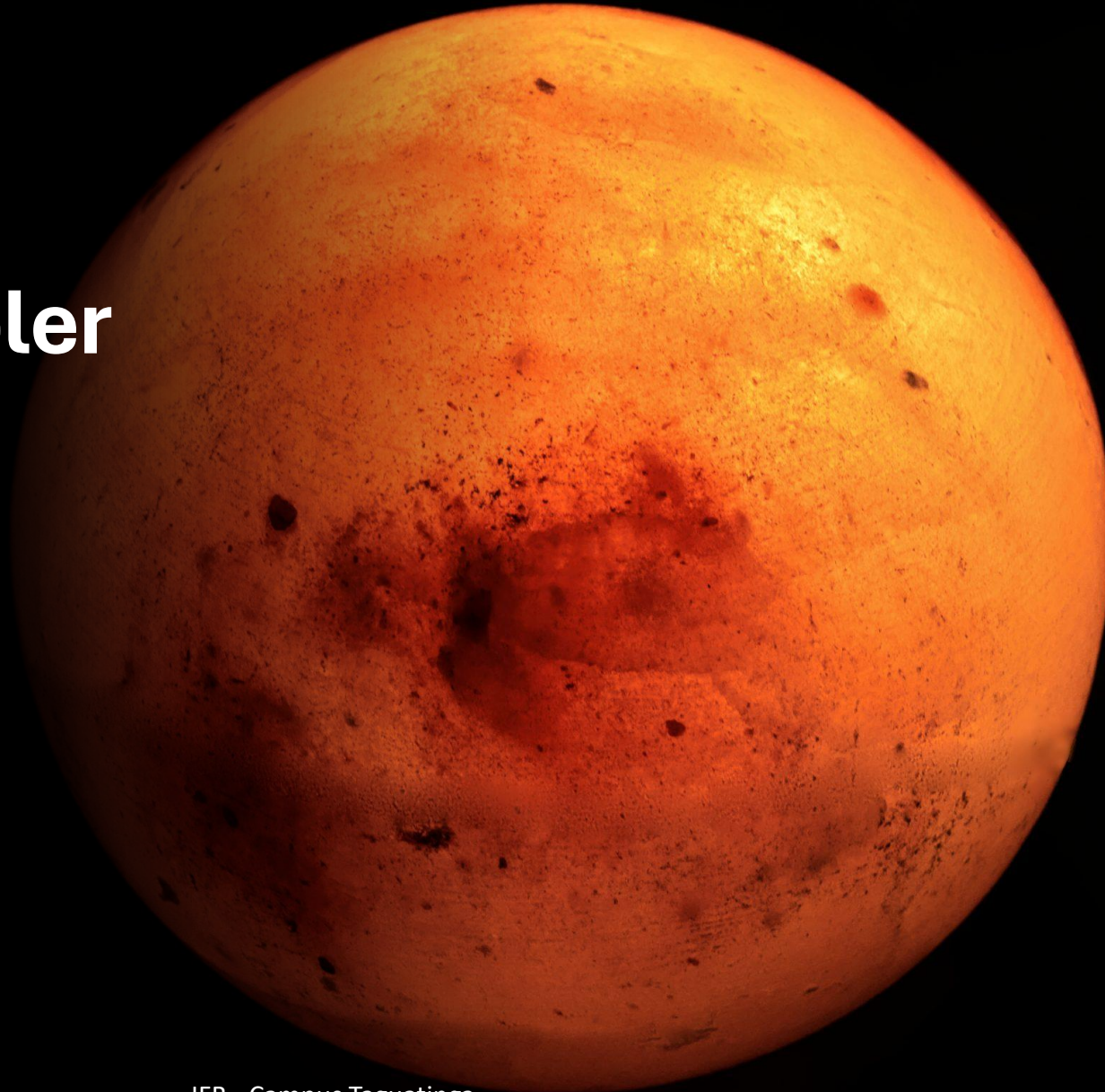
lb = estende o sinal.

lbu = estende zeros.

```
lb    $t1 , 0($t2) # $t1 = 0x00000011
lb    $t1 , 2($t2) # $t1 = 0xFFFFFFFF
lbu   $t1 , 0($t2) # $t1 = 0x00000011
lbu   $t1 , 2($t2) # $t1 = 0x000000CC
```



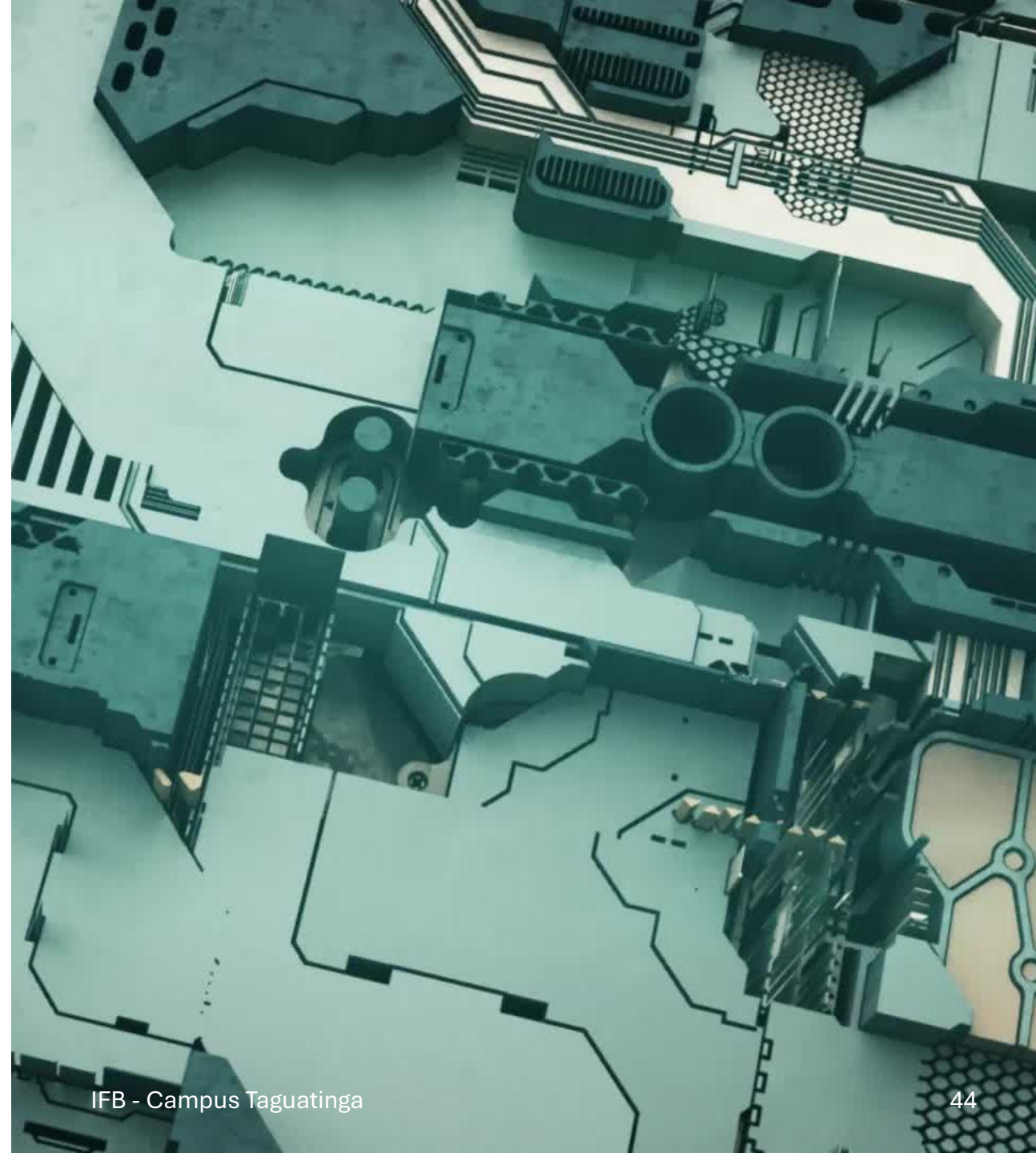
MARS (MIPS Assembler and Runtime Simulator)



MARS

O **MARS (MIPS Assembler and Runtime Simulator)** foi desenvolvido especificamente para ensino de Assembly MIPS e é baseado no material de Patterson & Hennessy.

Ele possui editor, assembler, simulador, visualização de registradores/memória e execução passo a passo.

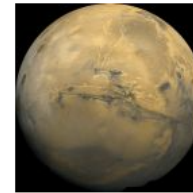


MARS 4.5 (2014) – Instalação

Entrar em:

<https://dpetersanderson.github.io/>

Fazer o download e abrir
(necessita ter o JAVA instalado).



- Home
- Features
- Download
- License
- Papers
- Help & Info

MARS (MIPS Assembler and Runtime Simulator)

An IDE for MIPS Assembly Language Programming

MARS is a lightweight interactive development environment (IDE) for programming in MIPS assembly language, intended for educational-level use with Patterson and Hennessy's *Computer Organization and Design*.



Feb. 2013: "MARS has been tested in the Softpedia labs using several industry-leading security solutions and found to be completely clean of adware/spyware components. ... Softpedia guarantees that MARS 4.3 is 100% FREE, which means it does not contain any form of malware, including spyware, viruses, trojans and backdoors."

[Download MARS from Softpedia](#) (version on Softpedia may lag behind the version on this page).

Download MARS 4.5 software! (Aug. 2014)

New for 4.0: new editor, featuring multiple files, context-sensitive input, and color-coding.

- Sample MIPS assembly program to run under MARS [Fibonacci.asm](#)

MARS features overview:

- GUI with point-and-click control and integrated editor
- Easily editable register and memory values, similar to a spreadsheet
- Display values in hexadecimal or decimal
- Command line mode for instructors to test and evaluate many programs easily
- Floating point registers, coprocessor1 and coprocessor2. Standard tool: bit-level view and edit of 32-bit floating point registers
- Variable-speed single-step execution
- "Tool" utility for MIPS control of simulated devices. Standard tool: Cache performance analysis tool
- Single-step backwards

MARS: Exemplo

.data

msg: .asciiz "Ola mundo!\n"

.text

main:

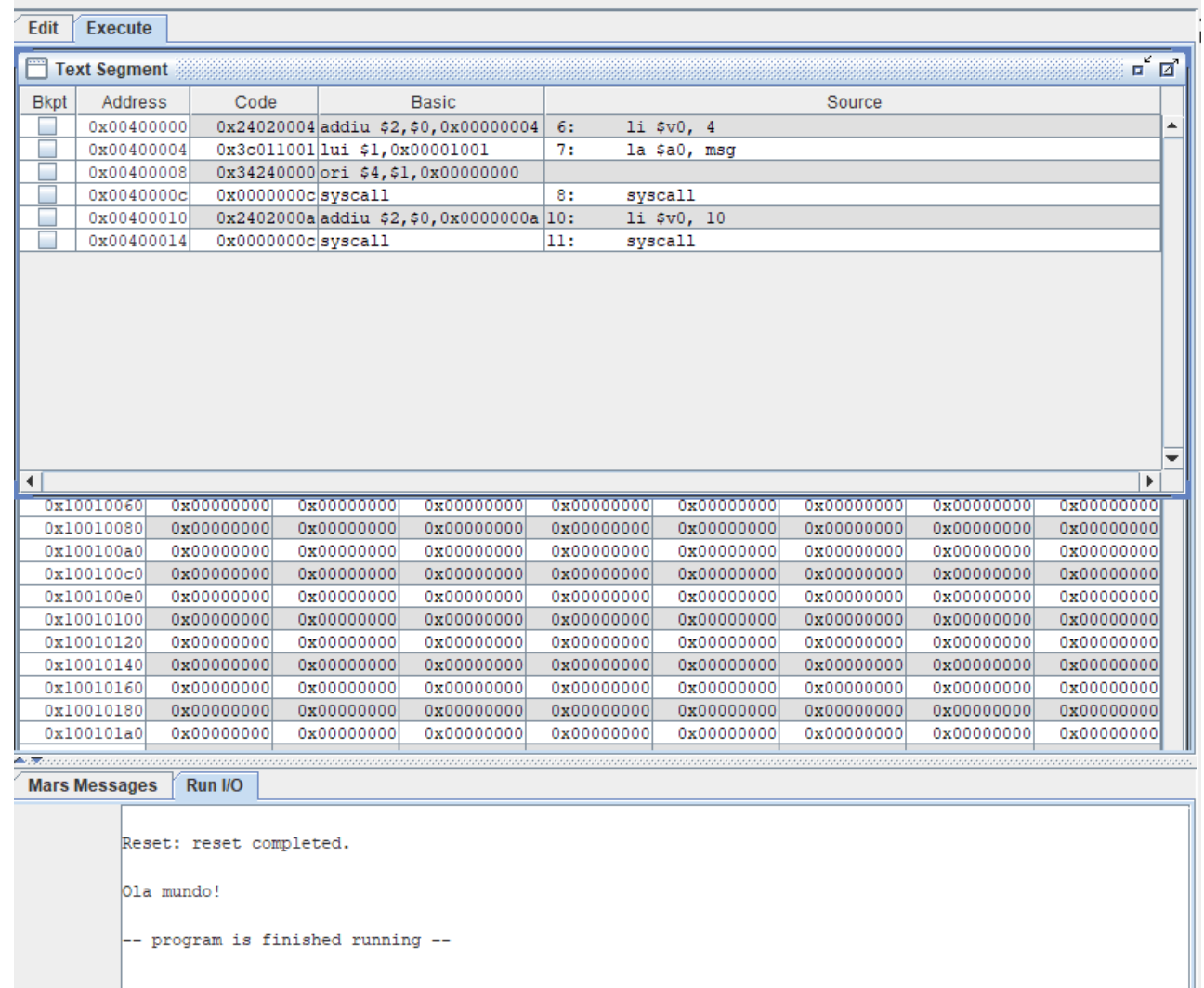
li \$v0, 4

la \$a0, msg

syscall

li \$v0, 10

syscall



Atividade Avaliativa 4

Assembler MIPS com MARS

Assembler MIPS com MARS

1. Instalar o MARS.
2. Fazer as 3 questões que seguem.
3. Gerar um relatório com os códigos escritos em Assembler.
4. Enviar no NEaD (atividade individual).

Questão 1

Escreva um programa em MIPS que inicialize dois registradores com valores inteiros:

- \$t0 = 25
- \$t1 = 40

O programa deverá **trocar os valores entre \$t0 e \$t1**, de modo que, ao final:

- \$t0 = 40
- \$t1 = 25

Questão 2

Escreva um programa MIPS que leia um número inteiro e determine se ele é **par ou ímpar**.

Exemplo

- Digite um numero: 27
- O numero é ímpar.

Dica: Usar a função “rem” para verificar o resto de uma função.

Questão 3

Escreva um programa MIPS que leia **três números inteiros** e determine qual deles é o maior.

Exemplo

- Digite o primeiro numero: 25
- Digite o segundo numero: 17
- Digite o terceiro numero: 31
- Maior: 31